

Artificial Intelligence and Applications

Unit 3: Adversarial Search & Logical Agents

**Vidyalankar School
of Information
Technology
Wadala (E),
Mumbai**
www.vsit.edu.in

Certificate

This is to certify that the e-book titled “Artificial Intelligence” comprises all elementary learning tools for a better understating of the relevant concepts. This e-book is comprehensively compiled as per the predefined eight parameters and guidelines.



Date: 20-8-2025

Ms. Kanchan Taksale
Department of IT & DS

 **DISCLAIMER:** The information contained in this e-book is compiled and distributed for educational purposes only. This e-book has been designed to help learners understand relevant concepts with a more dynamic interface. The compilers of this e-book and Vidyalankar School of Information technology give full and due credit to the authors of the contents, developers and all websites from wherever information has been sourced. We acknowledge our gratitude towards the websites YouTube, Wikipedia, and Google search engine. No commercial benefits are being drawn from this project.

Unit III: Adversarial Search & Logical Agents

Contents:

- Games, optimal decisions in games
- Alpha-beta pruning
- Stochastic games
- Partially observable games
- State-of-the-art game programs
- Knowledge base agents
- The Wumpus world
- Logic, propositional logic
- Propositional theorem proving
- Effective propositional model checking
- Agents based on propositional logic

Recommended Books

- Artificial Intelligence: A Modern Approach , Stuart Russel and Peter Norvig
- A First Course in Artificial Intelligence, Deepak Khemani

Prerequisites and Linking

Unit III	Pre- requisites					Linking
Adversarial Search and Logical Agents	Sem. I	Sem. II	Sem. III	Sem. IV	Sem. V	Sem. VI
	-	-	Python Programming	Core Java	-	Business Intelligence

Q.1 How the game can be formally defined as a kind of search problem?

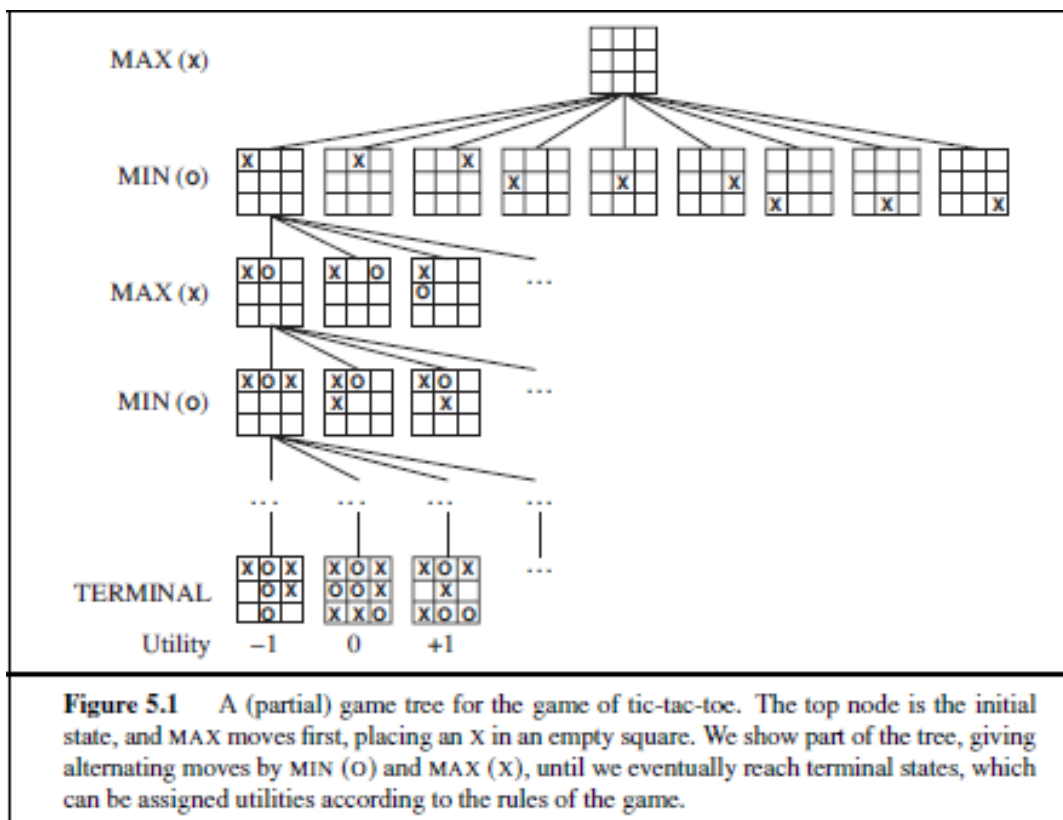
- a) We begin with a definition of the optimal move and an algorithm for finding it.
- b) **Pruning** allows us to ignore portions of the search tree that make no difference to the final choice, and heuristic **evaluation functions** allow us to approximate the true utility of a state without doing a complete search.

We first consider games with two players, whom we call MAX and MIN for reasons that will soon become obvious. MAX moves first, and then they take turns moving until the game is over.

- c) A game can be formally defined as a kind of search problem with the following elements:
 - **S0**: The **initial state**, which specifies how the game is set up at the start.
 - **PLAYER(s)**: Defines which player has the move in a state
 - **ACTIONS(s)**: Returns the set of legal moves in a state.
 - **RESULT(s, a)**: The **transition model**, which defines the result of a move.
 - **TERMINAL-TEST(s)**: A **terminal test**, which is true when the game is over and false otherwise. States where the game has ended are called **terminal states**.
 - **UTILITY(s, p)**: A **utility function** (also called an objective function or payoff function), defines the final numeric value for a game that ends in terminal state *s* for a player *p*. In chess, the outcome is a win, loss, or draw, with values +1, 0, or 1/2. Some games have a wider variety of possible outcomes; the payoffs in backgammon range from 0 to +192. A **zero-sum game** is (confusingly) defined as one where the total payoff to all players is the same for every instance of the game. Chess is zero-sum because every game has payoff of either 0 + 1, 1 + 0 or 1/2 + 1/2. “Constant-sum” would have been a better term, but zero-sum is traditional and makes sense if you imagine each player is charged an entry fee of 1/2.

Q.2 What is Game tree? Explain with example.

- a) The initial state, ACTIONS function, and RESULT function define the **game tree** for the game- tree where the nodes are game states and the edges are moves.
 - b) Eg. Game tree for tic-tac-toe.
 - c) From the initial state, MAX has nine possible moves.
 - d) Play alternates between MAX’s placing an X and MIN’s placing an O until we reach leaf nodes corresponding to terminal states such that one player has three in a row or all the squares are filled. The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are assumed to be good for MAX and bad for MIN (which is how the players get their names).
 - e) For tic-tac-toe the game tree is relatively small—fewer than $9! = 362,880$ terminal nodes. But for chess there are over 1040 nodes, so the game tree is best thought of as a theoretical construct that we cannot realize in the physical world.
 - f) But regardless of the size of the game tree, it is MAX’s job to search for a good move.
- SEARCH TREE** We use the term search tree for a tree that is superimposed on the full game tree, and examines enough nodes to allow a player to determine what move to make.



Q.3 What are the OPTIMAL DECISIONS IN GAMES?

- In a normal search problem, the optimal solution would be a sequence of actions leading to a goal state—a terminal state that is a win.
- In adversarial search, MIN has something to say about it. MAX therefore must find a contingent **strategy**, which specifies MAX's move in the initial state, then MAX's moves in the states resulting from every possible response by MIN, then MAX's moves in the states resulting from every possible response by MIN to *those* moves, and so on.
- This is exactly analogous to the AND–OR search algorithm with MAX playing the role of OR and MIN equivalent to AND.
- An optimal strategy leads to outcomes at least as good as any other strategy when one is playing an infallible opponent. We begin by showing how to find this optimal strategy.
- The possible moves for MAX at the root node are labeled a1, a2, and a3.
- The possible replies to a1 for MIN are b1, b2, b3, and so on.
- This particular game ends after one move each by MAX and MIN.
- The utilities of the terminal states in this game range from 2 to 14.
- MAX prefers to move to a state of maximum value, whereas MIN prefers a state of minimum value. So we have the following:

MINIMAX(s) =

UTILITY(s)		if TERMINAL-TEST(s)
max _{a ∈ Actions(s)}	MINIMAX(RESULT(s, a))	if PLAYER(s) = MAX
min _{a ∈ Actions(s)}	MINIMAX(RESULT(s, a))	if PLAYER(s) = MIN

Q.4 Explain min max algorithm with example.

- a) The **minimax algorithm** computes the minimax decision from the current state.
- b) It uses a simple recursive computation of the minimax values of each successor state, directly
- c) implementing the defining equations.
- d) The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are **backed up** through the tree as the recursion unwinds.
- e) The minimax algorithm performs a complete depth-first exploration of the game tree.
- f) The minimax algorithm performs a complete depth-first exploration of the game tree.
- g) If the maximum depth of the tree is m and there are b legal moves at each point, then the time complexity of the minimax algorithm is $O(b^m)$.
- h) The space complexity is $O(bm)$ for an algorithm that generates all actions at once, or $O(m)$ for an algorithm that generates actions one at a time.
- i) For real games, of course, the time cost is totally impractical, but this algorithm serves as the basis for the mathematical analysis of games and for more practical algorithms.

```
function MINIMAX-DECISION(state) returns an action
  return argmaxa ∈ ACTIONS(s) MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← -∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v

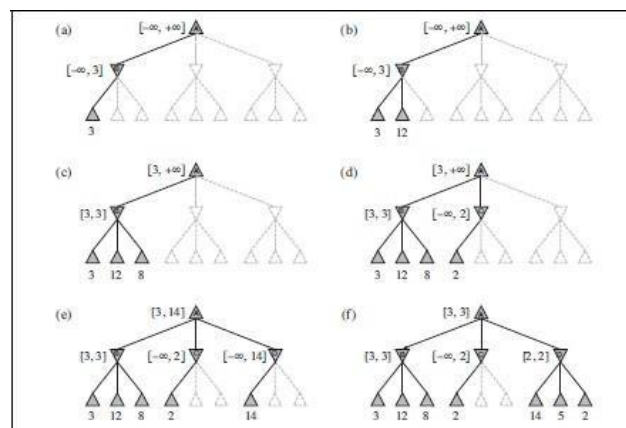
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

MINMAX Algorithm:

MINMAX Algorithm: <https://www.youtube.com/watch?v=djbYQuOJ-Zo>

Q.5 Explain alpha beta pruning algorithm with example

- The problem with minimax search is that the number of game states it has to examine is exponential in the depth of the tree.
- we can't eliminate the exponent, but it turns out we can effectively cut it in half. The trick is that it is possible to compute the correct minimax decision without looking at every node in the game tree.
- The particular technique we examine is called **alpha beta pruning**. When applied to a standard minimax tree, it returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.
- Another way to look at this is as a simplification of the formula for MINIMAX. Let the two unevaluated successors of node C in Figure 5.5 have values x and y . Then the value of the root node is given by
- $$\begin{aligned} \text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\ &= \max(3, \min(2, x, y), 2) \\ &= \max(3, z, 2) \text{ where } z = \min(2, x, y) \leq 2 \\ &= 3. \end{aligned}$$
- In other words, the value of the root and hence the minimax decision are *independent* of the values of the pruned leaves x and y .
- Alpha-beta pruning can be applied to trees of any depth, and it is often possible to prune entire subtrees rather than just leaves. The general principle is this: consider a node n somewhere in the tree, such that Player has a choice of moving to that node.
- Remember that minimax search is depth-first, so at any one time we just have to consider the nodes along a single path in the tree. Alpha-beta pruning gets its name from the following two parameters that describe bounds on the backed-up values that appear anywhere along the path:
- α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.
- β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.
- Alpha-beta search updates the values of α and β as it goes along and prunes the remaining branches at a node (i.e., terminates the recursive call) as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.



Step by Step: Alpha Beta Pruning

Alpha Beta Pruning: <https://www.youtube.com/watch?v=xBXHtz4Gbdo>

Q.6 Write a short note on Evaluation functions, Cutting off search.

Evaluation Function:

- a) An evaluation function returns an *estimate* of the expected utility of the game from a given Position.
- b) For centuries, chess players have developed ways of judging the value of a position because humans are even more limited in the amount of search they can do than are computer programs.
- c) It should be clear that the performance of a game-playing program depends strongly on the quality of its evaluation function.
- d) An inaccurate evaluation function will guide an agent toward positions that turn out to be lost.
- e) The evaluation function should order the *terminal* states in the same way as the true utility function: states that are wins must evaluate better than draws, which in turn must be better than losses.
- f) Otherwise, an agent using the evaluation function might err even if it can see ahead all the way to the end of the game.
- g) The computation must not take too long!
- h) For nonterminal states, the evaluation function should be strongly correlated with the actual chances of winning.
- i) Evaluation functions work by calculating various **features** of the state—for example, in chess, we would have features for the number of white pawns, black pawns, white queens, black queens, and so on.
- j) The features, taken together, define various *categories* or *equivalence classes* of states: the states in each category have the same values for all the features.
- k) Then a reasonable evaluation for states in the category is the **expected value**.

Cutting off Search:

- a) The most straightforward approach to controlling the amount of search is to set a fixed depth limit so that CUTOFF-TEST(state, depth) returns true for all depth greater than

some fixed depth d . The depth d is chosen so that a move is selected within the allocated time.

- b) The evaluation function should be applied only to positions that are **quiescent**— that is, unlikely to exhibit wild swings in value in the near future.
- c) This extra search is called a **quiescence search**; sometimes it is restricted to consider only certain types of moves, such as capture moves, that will quickly resolve the uncertainties in the position.
- d) The **horizon effect** is more difficult to eliminate. It arises when the program is facing an opponent's move that causes serious damage and is ultimately unavoidable, but can be temporarily avoided by delaying tactics. One strategy to mitigate the horizon effect is the **singular extension**, a move that is "clearly better" than all other moves in a given position.

Q.7 Define Forward pruning, Search versus lookup.

Forward Pruning

- a) It is also possible to do **forward pruning**, meaning that some moves at a given node are pruned immediately without further consideration. Clearly, most humans playing chess consider only a few moves from each position.
- b) One approach to forward pruning is **beam search**: on each ply, consider only a "beam" of the n best moves (according to the evaluation function) rather than considering all possible moves.
- c) The PROBCUT, or probabilistic cut, algorithm (Buro, 1995) is a forward-pruning version of alpha-beta search that uses statistics gained from prior experience to lessen the chance that the best move will be pruned. Alpha-beta search prunes any node that is *provably* outside the current (α, β) window.
- d) PROBCUT also prunes nodes that are *probably* outside the window. It computes this probability by doing a shallow search to compute the backed-up value v of a node and then using past experience to estimate how likely it is that a score of v at depth d in the tree would be outside (α, β) .

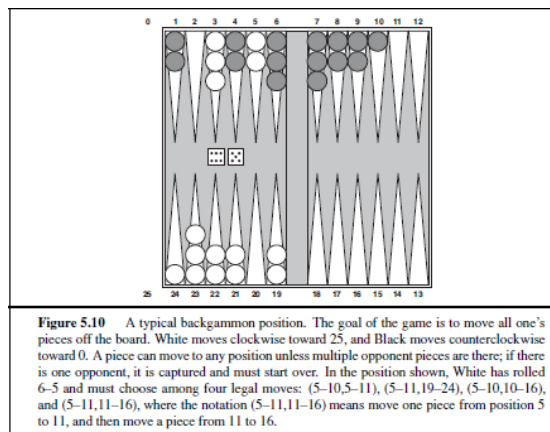
Search versus lookup

- a) The computer is mostly relying on the expertise of humans. The best advice of human experts on how to play each opening is copied from books and entered into tables for the computer's use. However, computers can also gather statistics from a database of previously played games to see which opening sequences most often lead to a win. In the early moves there are few choices, and thus much expert commentary and past games on which to draw. Usually after ten moves we end up in a rarely seen position, and the program must switch from table lookup to search
- b) Near the end of the game there are again fewer possible positions, and thus more chance to do lookup. But here it is the computer that has the expertise: computer analysis of endgames goes far beyond anything achieved by humans. A human can tell you the general strategy for playing a king-and-rook-versus-king (KRK) endgame: reduce the opposing king's mobility by squeezing it toward one edge of the board, using your king to prevent the opponent from escaping the squeeze
- c) A computer, on the other hand, can completely *SOLVE* the endgame by producing a **policy**, which is a mapping from every possible state to the best move in that state.
- d) Then we can just look up the best move rather than recompute it anew. How big will the KBNK lookup table be? It turns out there are 462 ways that two kings can be placed on the board without being adjacent.

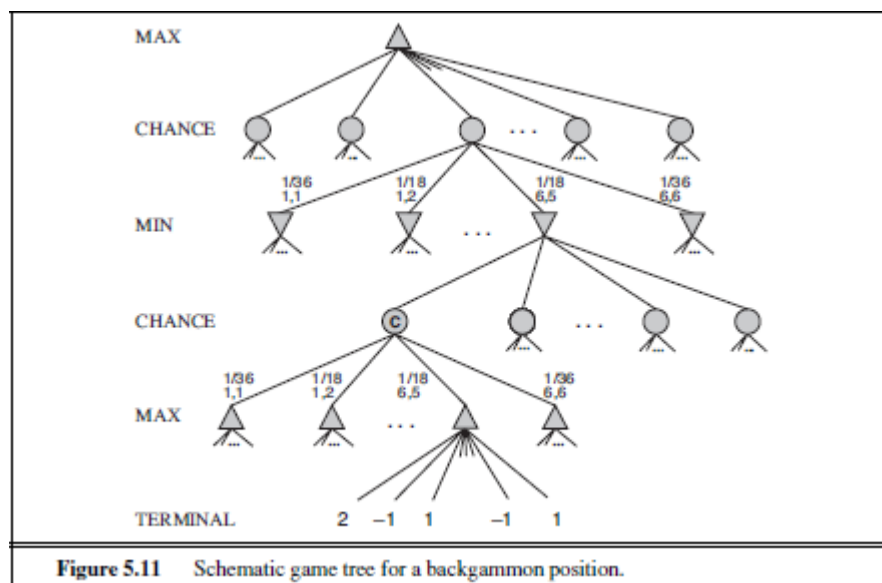
- e) After the kings are placed, there are 62 empty squares for the bishop, 61 for the knight, and two possible players to move next, so there are just $462 \times 62 \times 61 \times 2 = 3,494,568$ possible positions. Some of these are checkmates; mark them as such in a table. Then do a **retrograde** minimax search: reverse the rules of chess to do unmoves rather than moves.

Q.8 Write a short note on STOCHASTIC GAMES.

a) Many games mirror this unpredictability by including a random element, such as the throwing of dice. We call these **stochastic games**. Backgammon is a typical game that combines luck and skill. Dice are rolled at the beginning of a player's turn to determine the legal moves.



- b) Although White knows what his or her own legal moves are, White does not know what Black is going to roll and thus does not know what Black's legal moves will be. That means White cannot construct a standard game tree of the sort we saw in chess and tic-tac-toe. A game tree in backgammon must include **chance nodes** in addition to MAX and MIN nodes.



- c) In order to maximize the points or the average score, expectimax search algorithm can be used.
- d) In this we have “Max Node” as well as we have in the minimax search and “Chance nodes” like min nodes, except the outcome is uncertain. We can calculate expected utilities.

- e) A stochastic game can be considered as an extension of a markov decision process as there are multiple agents with possibly conflicting goals, and the joint actions of agents determine state transitions and rewards.

Q.9 Which are the different PARTIALLY OBSERVABLE GAMES?

- a) Chess has often been described as war in miniature, but it lacks at least one major characteristic of real wars, namely, **partial observability**. In the “fog of war,” the existence and disposition of enemy units is often unknown until revealed by direct contact. As a result, warfare includes the use of scouts and spies to gather information and the use of concealment and bluff to confuse the enemy.

Kriegspiel: Partially observable chess

- b) In *deterministic* partially observable games, uncertainty about the state of the board arises entirely from lack of access to the choices made by the opponent. This class includes children’s games such as Battleships (where each player’s ships are placed in locations hidden from the opponent but do not move) and Stratego (where piece locations are known but piece types are hidden).
- c) We will examine the game of **Kriegspiel**, a partially observable variant of chess in which pieces can move but are completely invisible to the opponent.
- d) The rules of Kriegspiel are as follows: White and Black each see a board containing only their own pieces. A referee, who can see all the pieces, adjudicates the game and periodically makes announcements that are heard by both players.
- e) White proposes to the referee any move that would be legal if there were no black pieces. If the move is in fact not legal (because of the black pieces), the referee announces “illegal.” In this case, White may keep proposing moves until a legal one is found—and learns more about the location of Black’s pieces in the process. Kriegspiel may seem terrifyingly impossible, but humans manage it quite well and computer programs are beginning to catch up.
- f) We can map Kriegspiel state estimation directly onto the partially observable, nondeterministic framework of Section 4.4 if we consider the opponent as the source of nondeterminism; that is, the RESULTS of White’s move are composed from the (predictable) outcome of White’s own move and the unpredictable outcome given by Black’s reply

Card Games

- g) Card games provide many examples of *stochastic* partial observability, where the missing
- h) information is generated randomly. For example, in many games, cards are dealt randomly at the beginning of the game, with each player receiving a hand that is not visible to the other players. Such games include bridge, whist, hearts, and some forms of poker.
- i) At first sight, it might seem that these card games are just like dice games: the cards are
- j) dealt randomly and determine the moves available to each player, but all the “dice” are rolled at the beginning! Even though this analogy turns out to be incorrect, it suggests an effective algorithm: consider all possible deals of the invisible cards; solve each one as if it were a fully observable game; and then choose the move that has the best outcome averaged over all the deals. Suppose that each deal s occurs with probability $P(s)$; then the move we want is

$$\operatorname{argmax}_a \sum_s P(s) \operatorname{MINIMAX}(\operatorname{RESULT}(s, a)) .$$

- k) we run exact MINIMAX if computationally feasible; otherwise, we run H-MINIMAX.
- l) Now, in most card games, the number of possible deals is rather large.

m) For games like whist and hearts, where there is no bidding or betting phase before play commences, each deal will be equally likely and so the values of $P(s)$ are all equal. For bridge, play is preceded by a bidding phase in which each team indicates how many tricks it expects to win. Since players bid based on the cards they hold, the other players learn more about the probability of each deal.

Q.10 Write a short note on Checkers, Othello, Backgammon, Go.

Checkers

- a) Jonathan Schaeffer and colleagues developed CHINOOK, which runs on regular PCs and uses alpha-beta search.
- b) Chinook defeated the long-running human champion in an
- c) abbreviated match in 1990, and since 2007 CHINOOK has been able to play perfectly by using
- d) alpha-beta search combined with a database of 39 trillion endgame positions.

Othello

- e) Othello also called Reversi, is probably more popular as a computer game than as a board game.
- f) It has a smaller search space than chess, usually 5 to 15 legal moves, but evaluation expertise had to be developed from scratch. In 1997, the LOGISTELLO program (Buro, 2002) defeated the human world champion, TakeshiMurakami, by six games to none. It is generally acknowledged that humans are no match for computers at Othello.

Backgammon

- g) Most work on backgammon has gone into improving the evaluation function. Gerry Tesauro (1992) combined reinforcement learning with neural networks to develop a remarkably accurate evaluator that is used with a search to depth 2 or 3.
- h) After playing more than a million training games against itself, Tesauro's program, TD-GAMMON, is competitive with top human players. The program's opinions on the opening moves of the game have in some cases radically altered the received wisdom.

Go

- i) Most popular board game in Asia.
- j) Programs are such that human champions are beginning to the challenge by machines, though the best humans still beat the best machines on the full board.

Q.11 Write a short note on KNOWLEDGE-BASED AGENTS.

- a) The central component of a knowledge-based agent is its **knowledge base**, or KB. A knowledge base is a set of **sentences**.
- b) Each sentence is expressed in a language called a **knowledge representation language** and represents some assertion about the world. Sometimes we dignify a sentence with the name **axiom**, when the sentence is taken as given without being derived from other sentences.
- c) Because of the definitions of TELL and ASK, however, the knowledge-based agent is not an arbitrary program for calculating actions. It is amenable to a description at the **knowledge level**, where we need specify only what the agent knows and what its goals are, in order to fix its behavior.
- d) Then we can expect it to cross the Golden Gate Bridge *because it knows that that will achieve its goal*. Notice that this analysis is independent of how the taxi works at the

implementation level. It doesn't matter whether its geographical knowledge is implemented as linked lists or pixel maps, or whether it reasons by manipulating strings of symbols stored in registers or by propagating noisy signals in a network of neurons.

- e) A knowledge-based agent can be built simply by TELLing it what it needs to know. Starting with an empty knowledge base, the agent designer can TELL sentences one by one until the agent knows how to operate in its environment. This is called the **declarative** approach to system building. In contrast, the **procedural** approach encodes desired behaviors directly as program code.
- f) We now understand that a successful agent often combines both declarative and procedural elements in its design, and that declarative knowledge can often be compiled into more efficient procedural code.

Q. 12 Define task environment for wumpus world problem.

- a) The **wumpus world** is a cave consisting of rooms connected by passageways. Lurking somewhere in the cave is the terrible wumpus, a beast that eats anyone who enters its room.
- b) The wumpus can be shot by an agent, but the agent has only one arrow. Some rooms contain bottomless pits that will trap anyone who wanders into these rooms
- c) The only mitigating feature of this bleak environment is the possibility of finding a heap of gold. Although the wumpus world is rather tame by modern computer game standards, it illustrates some important points about intelligence.
- d) The precise definition of the task environment is given, , by the PEAS description:

Performance measure:

- e) +1000 for climbing out of the cave with the gold, −1000 for falling into a pit or being eaten by the wumpus, −1 for each action taken and −10 for using up the arrow. The game ends either when the agent dies or when the agent climbs out of the cave.

Environment:

- f) A 4×4 grid of rooms. The agent always starts in the square labelled [1,1], facing to the right. The locations of the gold and the wumpus are chosen randomly, with a uniform distribution, from the squares other than the start square. In addition, each square other than the start can be a pit, with probability 0.2.

Actuators:

- g) The agent can move *Forward*, *TurnLeft* by 90°, or *TurnRight* by 90°. The agent dies a miserable death if it enters a square containing a pit or a live wumpus. (It is safe, albeit smelly, to enter a square with a dead wumpus.)
- h) If an agent tries to move forward and bumps into a wall, then the agent does not move. The action *Grab* can be used to pick up the gold if it is in the same square as the agent.
- i) The action *Shoot* can be used to fire an arrow in a straight line in the direction the agent is facing. The arrow continues until it either hits (and hence kills) the wumpus or hits a wall.
- j) The agent has only one arrow, so only the first *Shoot* action has any effect. Finally, the action *Climb* can be used to climb out of the cave, but only from square [1,1].

Sensors:

The agent has five sensors, each of which gives a single bit of information:

- In the square containing the wumpus and in the directly (not diagonally) adjacent squares, the agent will perceive a *Stench*.
- In the squares directly adjacent to a pit, the agent will perceive a *Breeze*.
- In the square where the gold is, the agent will perceive a *Glitter*.
- When an agent walks into a wall, it will perceive a *Bump*.

- When the wumpus is killed, it emits a woeful *Scream* that can be perceived anywhere in the cave.

The percepts will be given to the agent program in the form of a list of five symbols; for example, if there is a stench and a breeze, but no glitter, bump, or scream, the agent program will get [Stench, Breeze, None, None, None].

Q.13 What are the connectives of propositional logic?

- We now present a simple but powerful logic called **propositional logic**.
- Then we look at **entailment**—the relation between a sentence and another sentence that follows from it—and see how this leads to a simple algorithm for logical inference.
- The **syntax** of propositional logic defines the allowable sentences. The **atomic sentences** consist of a single **proposition symbol**.
- Each such symbol stands for a proposition that can be true or false.
- There are two proposition symbols with fixed meanings: True is the always-true proposition and False is the always-false proposition.
- Complex sentences** are constructed from simpler sentences, using parentheses and **logical connectives**. There are five connectives in common use:
 - $\neg$ (not). A sentence such as $\neg W_{1,3}$ is called the **negation** of $W_{1,3}$. A **literal** is either an atomic sentence (a **positive literal**) or a negated atomic sentence (a **negative literal**).
 - $\wedge$ (and). A sentence whose main connective is $\wedge$, such as $W_{1,3} \wedge P_{3,1}$, is called a **conjunction**; its parts are the **conjuncts**. (The $\wedge$ looks like an “A” for “And.”)
 - $\vee$ (or). A sentence using $\vee$, such as $(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$, is a **disjunction** of the **disjuncts** $(W_{1,3} \wedge P_{3,1})$ and $W_{2,2}$. (Historically, the $\vee$ comes from the Latin “vel,” which means “or.” For most people, it is easier to remember $\vee$ as an upside-down A.)
 - $\Rightarrow$ (implies). A sentence such as $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ is called an **implication** (or conditional). Its **premise** or **antecedent** is $(W_{1,3} \wedge P_{3,1})$, and its **conclusion** or **consequent** is $\neg W_{2,2}$. Implications are also known as **rules** or **if–then** statements. The implication RULES symbol is sometimes written in other books as $\supset$ or $\rightarrow$.
 - $\Leftrightarrow$ (if and only if). The sentence $W_{1,3} \Leftrightarrow \neg W_{2,2}$ is a **biconditional**. Some other books write this as $\equiv$.

Propositional Logic:

Propositional Logic: <https://www.youtube.com/watch?v=tpDU9UXqsUo>

Q.14 Write A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedence.

- The BNF grammar by itself is ambiguous; a sentence with several operators can be parsed by the grammar in multiple ways.
- To eliminate the ambiguity we define a precedence for each operator. The “not” operator ($\neg$) has the highest precedence, which means that in the sentence $\neg A A B$ the $\neg$ binds most tightly, giving us the equivalent of $(\neg A)AB$ rather than $\neg(AAB)$.
- When in doubt, use parentheses to make sure of the right interpretation. Square brackets mean the same thing as parentheses; the choice of square brackets or parentheses is solely to make it easier for a human to read a sentence.

<i>Sentence</i>	$\rightarrow$	<i>AtomicSentence</i> <i>ComplexSentence</i>
<i>AtomicSentence</i>	$\rightarrow$	<i>True</i> <i>False</i> <i>P</i> <i>Q</i> <i>R</i> ...
<i>ComplexSentence</i>	$\rightarrow$	(<i>Sentence</i>) [<i>Sentence</i>]
		$\neg$ <i>Sentence</i>
		<i>Sentence</i> $\wedge$ <i>Sentence</i>
		<i>Sentence</i> $\vee$ <i>Sentence</i>
		<i>Sentence</i> $\Rightarrow$ <i>Sentence</i>
		<i>Sentence</i> $\Leftrightarrow$ <i>Sentence</i>
OPERATOR PRECEDENCE	:	$\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Q.15 Write a Standard logical equivalence in PL.

$$\begin{aligned}(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge\end{aligned}$$

Q.16 Explain inference rules with examples.

- This section covers **inference rules** that can be applied to derive a **proof**—a chain of conclusions that leads to the desired goal. The best-known rule is called **Modus Ponens** (Latin for *mode that affirms*) and is written $\alpha \Rightarrow \beta, \alpha \vdash \beta$.
- The notation means that, whenever any sentences of the form $\alpha \Rightarrow \beta$ and α are given, then
- the sentence β can be inferred. For example, if $(\text{WumpusAhead } \text{AWumpusAlive}) \Rightarrow \text{Shoot}$ and $(\text{WumpusAhead } \text{AWumpusAlive})$ are given, then Shoot can be inferred. Another useful inference rule is **And-Elimination**, which says that, from a conjunction, any of the conjuncts can be inferred: $\alpha \wedge \beta \vdash \alpha$.
- Not all inference rules work in both directions like this. For example, we cannot run Modus Ponens in the opposite direction to obtain $\alpha \Rightarrow \beta$ and α from β .
- Let us see how these inference rules and equivalences can be used in the wumpus world.
- We start with the knowledge base containing R1 through R5 and show how to prove $\neg P_{1,2}$, that is, there is no pit in [1,2]. First, we apply biconditional elimination to R2 to obtain R6: $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$.
- Then we apply And-Elimination to R6 to obtain R7: $((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$.

Logical equivalence for contrapositives gives

$$R8: (\neg B_{1,1} \Rightarrow \neg (P_{1,2} \vee P_{2,1}))$$

Now we can apply Modus Ponens with R8 and the percept R4 (i.e., $\neg B_{1,1}$), to obtain

$$R9: \neg (P_{1,2} \vee P_{2,1})$$

Finally, we apply De Morgan's rule, giving the conclusion

$$R10: \neg P_{1,2} \wedge \neg P_{2,1}$$

That is, neither [1,2] nor [2,1] contains a pit.

- INITIAL STATE: the initial knowledge base.
- ACTIONS: the set of actions consists of all the inference rules applied to all the sentences that match the top half of the inference rule.

- **RESULT:** the result of an action is to add the sentence in the bottom half of the inference rule.
- **GOAL:** the goal is a state that contains the sentence we are trying to prove.

Q.17 Write a short note on Forward and backward chaining.

- The forward-chaining algorithm $PL\text{-}FC\text{-}ENTAILS?(KB, q)$ determines if a single proposition symbol q —the query—is entailed by a knowledge base of definite clauses. It begins from known facts (positive literals) in the knowledge base. If all the premises of an implication are known, then its conclusion is added to the set of known facts.
- This process continues until the query q is added or until no further inferences can be made.
- Algorithm

```

function  $PL\text{-}FC\text{-}ENTAILS?(KB, q)$  returns true or false
  inputs:  $KB$ , the knowledge base, a set of propositional definite clauses
            $q$ , the query, a proposition symbol
   $count \leftarrow$  a table, where  $count[c]$  is the number of symbols in  $c$ 's premise
   $inferred \leftarrow$  a table, where  $inferred[s]$  is initially false for all symbols
   $agenda \leftarrow$  a queue of symbols, initially symbols known to be true in  $KB$ 

  while  $agenda$  is not empty do
     $p \leftarrow POP(agenda)$ 
    if  $p = q$  then return true
    if  $inferred[p] = false$  then
       $inferred[p] \leftarrow true$ 
      for each clause  $c$  in  $KB$  where  $p$  is in  $c.PREMISE$  do
        decrement  $count[c]$ 
        if  $count[c] = 0$  then add  $c.CONCLUSION$  to  $agenda$ 
  return false

```

Q.18 Write a short note on backtracking algorithm.

```

function  $BACKTRACKING\text{-}SEARCH(csp)$  returns a solution, or failure
  return  $BACKTRACK(\{\}, csp)$ 

function  $BACKTRACK(assignment, csp)$  returns a solution, or failure
  if  $assignment$  is complete then return  $assignment$ 
   $var \leftarrow SELECT\text{-}UNASSIGNED\text{-}VARIABLE(csp)$ 
  for each  $value$  in  $ORDER\text{-}DOMAIN\text{-}VALUES(var, assignment, csp)$  do
    if  $value$  is consistent with  $assignment$  then
      add  $\{var = value\}$  to  $assignment$ 
       $inferences \leftarrow INFERENCES(csp, var, value)$ 
      if  $inferences \neq failure$  then
        add  $inferences$  to  $assignment$ 
         $result \leftarrow BACKTRACK(assignment, csp)$ 
        if  $result \neq failure$  then
          return  $result$ 
      remove  $\{var = value\}$  and  $inferences$  from  $assignment$ 
  return failure

```

The term **backtracking search** is used for a depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign.

If an inconsistency is detected, then $BACKTRACK$ returns failure, causing the previous call to try another value.

Notice that $BACKTRACKING\text{-}SEARCH$ keeps only a single representation of a state and alters that representation rather than creating new ones.

Q.19 What are the tricks that enable SAT solvers to scale up to large problems?

- a) The tricks that enable SAT solvers to scale up to large problems. It is interesting that most of these tricks are in fact rather general, and we have seen them before in other guises:
- b) **Component analysis** : As DPLL assigns truth values to variables, the set of clauses may become separated into disjoint subsets, called **components**, that share no unassigned variables. Given an efficient way to detect when this occurs, a solver can gain considerable speed by working on each component separately.
- c) **Variable and value ordering**: Our simple implementation of DPLL uses an arbitrary variable ordering and always tries the value *true* before *false*. The **degree heuristic** suggests choosing the variable that appears most frequently over all remaining clauses.
- d) **Intelligent backtracking** : Many problems that cannot be solved in hours of run time with chronological backtracking can be solved in seconds with intelligent backtracking that backs up all the way to the relevant point of conflict. All SAT solvers that do intelligent backtracking use some form of **conflict clause learning** to record conflicts so that they won't be repeated later in the search.
- e) **Random restarts** : Sometimes a run appears not to be making progress. In this case, we can start over from the top of the search tree, rather than trying to continue. After restarting, different random choices (in variable and value selection) are made.
- f) **Clever indexing** (as seen in many algorithms): The speedup methods used in DPLL itself, as well as the tricks used in modern solvers, require fast indexing of such things as "the set of clauses in which variable X_i appears as a positive literal." This task is complicated by the fact that the algorithms are interested only in the clauses that have not yet been satisfied by previous assignments to variables, so the indexing structures must be updated dynamically as the computation proceeds.

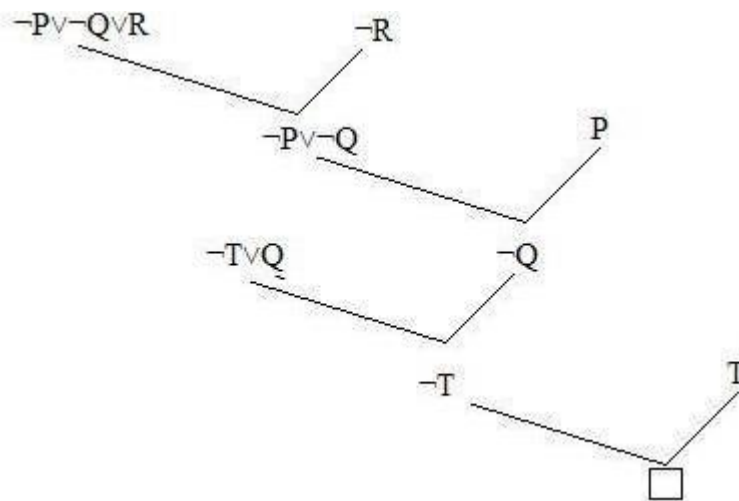
Q.20 Explain Concept of knowledge base with example.

- a) The central component of a knowledge-based agent is its **knowledge base**, or KB. A knowledge base is a set of **sentences**.
- b) Each sentence is expressed in a language called a **knowledge representation language** and represents some assertion about the world. Sometimes we dignify a sentence with the name **axiom**, when the sentence is taken as given without being derived from other sentences.
- c) Because of the definitions of TELL and ASK, however, the knowledge-based agent is not an arbitrary program for calculating actions. It is amenable to a description at the **knowledge level**, where we need specify only what the agent knows and what its goals are, in order to fix its behavior.
- d) Then we can expect it to cross the Golden Gate Bridge *because it knows that that will achieve its goal*. Notice that this analysis is independent of how the taxi works at the **implementation level**. It doesn't matter whether its geographical knowledge is implemented as linked lists or pixel maps, or whether it reasons by manipulating strings of symbols stored in registers or by propagating noisy signals in a network of neurons.
- e) A knowledge-based agent can be built simply by TELLing it what it needs to know. Starting with an empty knowledge base, the agent designer can TELL sentences one by one until the agent knows how to operate in its environment. This is called the **declarative** approach to system building. In contrast, the **procedural** approach encodes desired behaviors directly as program code.

- f) We now understand that a successful agent often combines both declarative and procedural elements in its design, and that declarative knowledge can often be compiled into more efficient procedural code.

Q.20 Explain Concept of Resolution theorem with example.

Resolution is a rule of inference leading to a refutation theorem-proving technique for sentences in propositional logic and first-order logic. In other words, iteratively applying the resolution rule in a suitable way allows for telling whether a propositional formula is satisfiable and for proving that a first-order formula is unsatisfiable.



GQs

1. Define game formally with important elements.
2. Explain how importance of optimal decisions in games in brief.
3. Explain following w.r.t. optimal decision in games. 1. Ply 2. Minimax
4. State and explain Minimax algorithm.
5. State and explain alpha beta pruning algorithm and Monte Carlo simulation w.r.t. it.
6. What is meant by strategy? Explain stochastic strategy and pure strategy.
7. Explain contribution of A.I. in algorithm of stochastic game.
8. Explain state of the art game programs with various games.
9. Write a short note on knowledge-based agent.
10. Explain wumpus world game with diagram and agent program of it.
11. Write a short note on promotional logic.
12. Explain Backus-Naur Form grammar of sentences along with operator precedence.
13. Explain semantics and atomic sentences w.r.t. A.I.
14. Explain how Propositional Logic is used to solve wumpus world problem.
15. Write a short note on standard logical equivalences for arbitrary sentences of propositional logic.
16. Explain Propositional Theorem Proving for WUMPUS World.
17. What is meant by monotonicity? Explain it w.r.t. propositional theorem.
18. What is meant by conjunctive normal form? Explain.
19. Write resolution algorithm and ground resolution algorithm for propositional logic.
20. Explain the difference between horn and definite clauses.

21. Explain Forward-chaining And Backward chaining in brief.
22. Write a short note on Backtracking Algorithm / DPLL algorithm.
23. What is meant by Satisfiability Problem? Explain tricks used for SAT solver.
24. Explain WALKSAT algorithm in detail.
25. Explain Landscape of Random SAT Problems.
26. What is meant by Satisfiability Threshold Conjecture? Explain with diagram.
27. Explain role of agent and agent's percept for the solution of Wumpus world problem.
28. Explain following Terms w.r.t. The Agent's Percept. 1. A temporal Variables 2. Locality 3. Inferential Frame Problem 4. Qualification Problem
29. What is meant by hybrid agent? Explain.
30. Explain Logical State Estimation in detail.
31. Write Hybrid Wumpus Agent algorithm with percept.
32. Explain how to make plans by logical inference to solve problem.
33. State and explain SATPlan Algorithm Translated In CNF.
34. Explain in brief about resolution theorem.
35. Write a note on Kriegspiel's Partially observable chess.
36. List and explain the elements used to define the game formally.

Fill in the Blanks

1. **Pruning** allows us to ignore portions of the search tree that make no difference to the final choice, and heuristic **evaluation functions** allow us to approximate the true utility of a state without doing a complete search.
2. A **utility function** also called an objective function or payoff function.
3. The **minimax algorithm** computes the minimax decision from the current state.
4. An **evaluation function** returns an *estimate* of the expected utility of the game from a given Position.
5. It is also possible to do **forward pruning**, meaning that some moves at a given node are pruned immediately without further consideration.
6. Many games mirror this unpredictability by including a random element, such as the throwing of dice. We call these **stochastic games**.
7. The central component of a knowledge-based agent is its **knowledge base**, or **KB**. A knowledge base is a set of **sentences**.
8. Complex sentences are constructed from simpler sentences, using parentheses and **logical connectives**.
9. **Resolution** is a rule of inference leading to a refutation theorem-proving technique for sentences in propositional logic and first-order logic.

Multiple Choice Questions

1. General games involves _____
 - a) Single-agent
 - b) Multi-agent
 - c) Neither Single-agent nor Multi-agent
 - d) Only Single-agent and Multi-agent
2. Adversarial search problems uses _____
 - a) Competitive Environment
 - b) Cooperative Environment

- c) Neither Competitive nor Cooperative Environment
- d) Only Competitive and Cooperative Environment

3. Mathematical game theory, a branch of economics, views any multi-agent environment as a game provided that the impact of each agent on the others is “significant,” regardless of whether the agents are cooperative or competitive.

- a) True
- b) False

4. Zero sum games are the one in which there are two agents whose actions must alternate and in which the utility values at the end of the game are always the same.

- a) True
- b) False

5. Zero sum game has to be a _____ game.

- a) Single player
- b) Two player
- c) Multiplayer
- d) Three player

6. A game can be formally defined as a kind of search problem with the following components.

- a) Initial State
- b) Successor Function
- c) Terminal Test
- d) All of the mentioned

7. The initial state and the legal moves for each side define the _____ for the game.

- a) Search Tree
- b) Game Tree
- c) State Space Search
- d) Forest

8. General algorithm applied on game tree for making decision of win/lose is

- a) DFS/BFS Search Algorithms
- b) Heuristic Search Algorithms
- c) Greedy Search Algorithms
- d) MIN/MAX Algorithms

9. The minimax algorithm computes the minimax decision from the current state. It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations. The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are backed up through the tree as the recursion unwinds.

- a) True
- b) False

10. What is the complexity of minimax algorithm?

- a) Same as of DFS

- b) Space – bm and time – bm
- c) Time – bm and space – bm
- d) Same as BFS

11. Which function is used to calculate the feasibility of whole game tree?

- a) Evaluation function
- b) Transposition
- c) Alpha-beta pruning
- d) All of the mentioned

12. Which is identical to the closed list in Graph search?

- a) Hill climbing search algorithm
- b) Depth-first search
- c) Transposition table
- d) None of the mentioned

13. What is called as transposition table?

- a) Hash table of next seen positions
- b) Hash table of previously seen positions
- c) Next value in the search
- d) None of the mentioned

14. How the effectiveness of the alpha-beta pruning gets increased?

- a) Depends on the nodes
- b) Depends on the order in which they are executed
- c) All of the mentioned
- d) None of the mentioned

15. Where does the values of alpha-beta search get updated?

- a) Along the path of search
- b) Initial state itself
- c) At the end
- d) None of the mentioned

16. Which value is assigned to alpha and beta in the alpha-beta pruning?

- a) Alpha = max
- b) Beta = min
- c) Beta = max
- d) Both Alpha = max & Beta = min

17. Which search is similar to minimax search?

- a) Hill-climbing search
- b) Depth-first search
- c) Breadth-first search
- d) All of the mentioned

18. To which depth does the alpha-beta pruning can be applied?

- a) 10 states
- b) 8 States
- c) 6 States

d) Any depth

19. Which values are independent in minimax search algorithm?

- a) Pruned leaves x and y
- b) Every states are dependant
- c) Root is independant
- d) None of the mentioned

20. Which approach is to pretend that a pure divide and conquer algorithm will work?

- a) Goal independence
- b) Subgoal independence
- c) Both Goal & Subgoal independence
- d) None of the mentioned

Previous Year University Questions

1. Explain the working mechanism of min-max algorithm. Oct 2018
2. Explain in brief about resolution theorem. Oct 2018
3. Write a note on Kriegspiel's Partially observable chess. Oct 2018
4. Explain in brief about knowledge base agent. Oct 2018
or
5. Explain the concept of knowledge base with example. May 2019
6. Explain the syntax for propositional logic. Oct 2018
7. Write a note on Wumpus world problem. Oct 2018
8. List and explain the elements used to define the game formally. May 2019
9. Write the minmax algorithm. Explain in short. May 2019
10. Explain alpha beta pruning with suitable example. May 2019
11. Write the connectives used to form complex sentence of propositional logic. Give example for each. May 2019
12. Write a short note on propositional thermo proving. May 2019